

MATH 231 – Linear Algebra I

The Computational Unit, Part 1 ■ Counting Operations

Kiely Hall 326 ■ Instructor: Kevin Bedoya

About this document. This opens a new unit, on the computational side of the subject: how much work a calculation actually costs, how numbers are really stored, and how the algorithms of this course are built and analysed.

A common confusion (Section numbers restart). This unit numbers its sections from 1 again — they do *not* continue from the vector unit. So a reference to “§1.3” below means §1.3 *of this unit*. Whenever we point back at the earlier material we will say so explicitly, as in “the vector unit, §11.3”. Both units have a §11.3, and they are different sections.

Within this unit, though, the numbering does run continuously from part to part:

Part	Sections	Subject
1	§1–§4	operation counting, Big-O, floating point, error
2	§5–§10	k -nearest neighbours
3	§11–§15	k -means clustering

Assumed from the vector unit. Vectors in $\mathbb{R}^n$ and componentwise addition (§7 there); the dot product $\langle \mathbf{u}, \mathbf{v} \rangle = \sum_i u_i v_i$ (§11.3); the Euclidean norm (§2); the projection $\text{proj}_{\mathbf{u}}(\mathbf{v})$ (§12.4); the identity $\langle \mathbf{u}, \mathbf{u} \rangle = \|\mathbf{u}\|^2$ (§11.6); orthogonality and cosine similarity (§11.4, §11.7); and linear dependence (§18.4).

What you should be able to do afterwards. Say what an *operation* is; count exactly how many operations vector addition, the dot product, the norm and the projection require in $\mathbb{R}^n$; recognise redundant work in a formula and remove it; state a count’s Big-O complexity and prove it from the definition; convert a decimal number to binary and encode it as an IEEE 754 float; read off the precision, the exponent range and the machine precision of a format; measure absolute and relative error for scalars and for vectors; and say what a FLOP is and what rounding does to one.

1. Counting operations

1.1. A first count: adding two vectors

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and set $\mathbf{w} = \mathbf{u} + \mathbf{v}$. Here is a question we have never asked before:

How many calculations must a machine perform in order to produce $\mathbf{w}$?

Before answering, one condition. Addition of vectors — like every vector operation in this course — is defined only when both inputs have the *same* dimension. There is no sum of a vector in $\mathbb{R}^3$ and a vector in $\mathbb{R}^5$: the definition matches up first coordinate with first, second with second, and would run out. So we may take both to have exactly n coordinates.

Now count. Addition is componentwise, so

$$\mathbf{w} = [u_1 + v_1, u_2 + v_2, \dots, u_n + v_n].$$

Each slot of the answer requires exactly one addition of two real numbers, and there are n slots. Nothing else happens. So producing $\mathbf{w}$ costs

$$n \text{ operations}$$

and the same count holds for $\mathbf{u} - \mathbf{v}$, which differs only in replacing each addition by a subtraction.

1.2. What counts as an operation

Definition 1.1 (Operation). In a computational setting, an **operation** is a single calculation carried out on scalar quantities by one operator — one addition, one subtraction, one multiplication, one division, one square root.

Two pieces of vocabulary we will use constantly: a scalar addition is an **add**, and a scalar multiplication is a **mult**. So the count above reads: vector addition in $\mathbb{R}^n$ costs n adds.

Remark. Counting every operation as costing “one” is a simplification, and a deliberate one. On real hardware a multiplication is somewhat slower than an addition, a division is several times slower again, and a square root slower still. We ignore that for now because the first thing worth knowing about a computation is *how the count grows with n* , and that pattern is the same whichever weights you attach. We will revisit the assumption when it starts to matter.

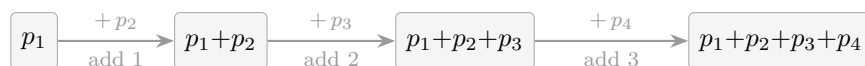
1.3. The dot product

Now take $\langle \mathbf{u}, \mathbf{v} \rangle$. Write out what the definition asks for:

$$\langle \mathbf{u}, \mathbf{v} \rangle = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n.$$

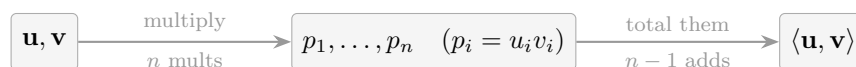
Two distinct stages are visible. First, n products $u_i v_i$ must be formed — that is n mults. Then those n numbers must be added into a single total.

How many adds does it take to total n numbers? Not n . Adding is a *binary* operation, so each add consumes two numbers and returns one, reducing the count of outstanding numbers by exactly one. Starting with n numbers and finishing with 1 therefore takes $n - 1$ adds.



4 numbers, 3 adds — in general n numbers, $n - 1$ adds

So the dot product runs in two stages:



Adding the two stages,

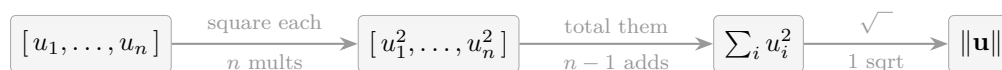
$$\langle \mathbf{u}, \mathbf{v} \rangle \text{ costs } n + (n - 1) = 2n - 1 \text{ operations}$$

1.4. The norm

Next, $\|\mathbf{u}\|$. Written out term by term,

$$\|\mathbf{u}\| = (u_1^2 + u_2^2 + \cdots + u_n^2)^{1/2}.$$

Now three stages. Each u_i^2 is the product $u_i \cdot u_i$, so squaring all n coordinates is n mults. Totalling those n squares is $n - 1$ adds, by the counting argument of §1.3. And finally one square root is taken, of a single scalar — one operation, no matter how large n is, because by that point only one number is left.



Adding the three stages,

$$\|\mathbf{u}\| \text{ costs } n + (n - 1) + 1 = 2n \text{ operations}$$

Remark. It is worth noticing where the 1 sits. The square root is applied *once*, at the very end, to a scalar — not once per coordinate. Miscounting this is the most common error in a first operation count: the stages of a computation operate on different numbers of quantities, and you must track how many are still in play at each stage.

1.5. The projection

Now something with several moving parts. Recall from the vector unit, §12.4,

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|} \cdot \frac{\mathbf{u}}{\|\mathbf{u}\|}.$$

The hard work is already done: we know what a dot product and a norm cost. What remains is to take the formula apart and add up the pieces, being careful at each step about whether the quantity involved is a scalar or a vector.

1. $\langle \mathbf{u}, \mathbf{v} \rangle$ — a dot product, so $2n - 1$ operations.
2. $\|\mathbf{u}\|$ — a norm, so $2n$ operations. The formula asks for it *twice*, once in each factor, so as written that is $4n$.
3. $\langle \mathbf{u}, \mathbf{v} \rangle / \|\mathbf{u}\|$ — careful here. Both of these are *scalars*, so this is a single scalar division: 1 operation.
4. $\mathbf{u} / \|\mathbf{u}\|$ — here $\mathbf{u}$ is a *vector*, and dividing it by a scalar is componentwise. That is n divisions, so n operations.
5. Finally the scalar from step 3 multiplies the vector from step 4, again componentwise: n mults, so n operations.

Adding them up,

$$(2n - 1) + 4n + 1 + n + n = 8n,$$

so the formula *as written* costs $8n$ operations.

1.6. Doing it better

Is there anything wrong with that count? Mathematically, no — $8n$ is the honest cost of evaluating the expression exactly as it appears. But to anyone who has written code, the problem is glaring.

First saving: stop computing the same thing twice

Why did we compute $\|\mathbf{u}\|$ *twice*? The two occurrences are the same number. Compute it once, keep it, and use it in both places. That deletes one of the two $2n$ charges:

$$(2n - 1) + 2n + 1 + n + n = 6n.$$

Nothing about the mathematics changed — only the bookkeeping. This is the single most common optimisation there is, and it has a name in programming: *common subexpression elimination*.

Second saving: do not take a square root you are going to square

Look at what $\|\mathbf{u}\|$ is used for. It appears twice, and both times it divides — so what the formula really uses is $\|\mathbf{u}\|^2$. And by the vector unit's §11.6,

$$\|\mathbf{u}\|^2 = \langle \mathbf{u}, \mathbf{u} \rangle,$$

which is a dot product: $2n - 1$ operations, with *no square root at all*. Rewriting the projection in the equivalent form already given in §12.4,

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|^2} \mathbf{u},$$

the count becomes

$$\underbrace{(2n - 1)}_{\langle \mathbf{u}, \mathbf{v} \rangle} + \underbrace{(2n - 1)}_{\langle \mathbf{u}, \mathbf{u} \rangle} + \underbrace{1}_{\text{one scalar division}} + \underbrace{n}_{\text{scale } \mathbf{u}} = 5n - 1.$$

Notice the third saving hiding in that line. The old version made two separate passes over $\mathbf{u}$ — dividing by the norm, then multiplying by the scalar — costing $n + n$. Combining the two scalars into one *before* touching the vector reduces that to a single pass of n mults.

Version	Cost	What changed
As written	$8n$	—
Reuse $\ \mathbf{u}\ $	$6n$	compute the norm once, not twice
Use $\ \mathbf{u}\ ^2 = \langle \mathbf{u}, \mathbf{u} \rangle$	$5n - 1$	no square root; one pass over $\mathbf{u}$

For $n = 1000$ that is 8000 operations against 4999 — the same answer, at a little over half the cost. And the square root, which we noted in §1.2 is among the slowest operations available, has been removed entirely.

1.7. Why count at all

A fair question at this point is why any of this matters. The formulas are correct either way, and a modern machine will evaluate all three versions faster than you can notice.

The reason is that we are stepping out of mathematics and into computation. Until now every object in this course has been exact and every operation free: $\mathbb{R}^n$ is infinite, a real number carries infinitely many digits, and writing $\|\mathbf{u}\|$ costs nothing because writing is not computing. A machine has none of that. It has finitely many digits per number, finitely many operations per second, and finite memory, and every one of those limits will eventually bind.

The computations here are small, and modern hardware and libraries are heavily optimised for exactly them. But the habit is what matters: taking a formula apart, asking what each stage costs, and noticing when the same quantity is being computed twice. Do that on a projection and you save a square root. Do it on an algorithm that runs a projection a million times inside a loop, and the difference is the difference between a program that finishes and one that does not.

Looking ahead. Two threads start here. First, we have been counting *exactly*, producing answers like $5n - 1$; usually the constants and the -1 do not matter and only the growth rate does, which is what *asymptotic* notation captures — the reason $5n - 1$ and $8n$ are “the same” in a sense worth making precise. Second, we have assumed a machine stores real numbers. It does not; it stores *floating-point* numbers, of which there are finitely many, and arithmetic on them is not quite the arithmetic of $\mathbb{R}$. Both are next.

1.8. Exercises

In each case, count the operations required, and keep the count *as low as possible* — part of the exercise is noticing what the given information lets you skip.

E1. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{200}$ be orthonormal. Count the operations needed to compute $\text{proj}_{\mathbf{u}}(\mathbf{v})$.

E2. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{50}$ with $\cos(\mathbf{u}, \mathbf{v}) = 1$. Count the operations needed to compute $\|\mathbf{u} - \mathbf{v}\|$.

E3. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{25}$ be nonzero and linearly dependent. Count the operations needed to compute $\cos(\mathbf{u}, \mathbf{v})$.

Answer key

E1. *Zero operations.* “Orthonormal” means both vectors are unit length *and* orthogonal, so $\langle \mathbf{u}, \mathbf{v} \rangle = 0$ is given. Then

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|^2} \mathbf{u} = \frac{0}{1} \mathbf{u} = \mathbf{0},$$

and the answer is the zero vector, written down without arithmetic. Ignoring the hypothesis and evaluating the optimised formula would have cost $5n - 1 = 999$ operations to arrive at the same $\mathbf{0}$. The lesson is that recognising structure beats computing.

E2. $3n = 150$ operations, and the hypothesis does not help. Directly: $\mathbf{u} - \mathbf{v}$ costs $n = 50$ subtractions, and taking the norm of the result costs $2n = 100$, for $3n = 150$ in total.

It is worth seeing why the given information is no use. Since $\cos(\mathbf{u}, \mathbf{v}) = 1$ the two vectors point the same way, and expanding as in the vector unit’s §13 gives $\|\mathbf{u} - \mathbf{v}\|^2 = \|\mathbf{u}\|^2 - 2\langle \mathbf{u}, \mathbf{v} \rangle + \|\mathbf{v}\|^2 = (\|\mathbf{u}\| - \|\mathbf{v}\|)^2$, so $\|\mathbf{u} - \mathbf{v}\| = \left| \|\mathbf{u}\| - \|\mathbf{v}\| \right|$. That is a genuine shortcut in appearance, but it costs two norms and a subtraction: $2n + 2n + 1 = 4n + 1 = 201$, which is *more* than 150. A shortcut is only a shortcut if it is cheaper.

E3. 1 *operation*. Linearly dependent and nonzero means parallel (vector unit, §18.4), so $\mathbf{v} = \alpha \mathbf{u}$ for some $\alpha \neq 0$, and the angle between them is 0 or π . Hence $\cos(\mathbf{u}, \mathbf{v}) = \pm 1$, with the sign being the sign of α . To find that sign, pick any index i with $u_i \neq 0$ and form $\alpha = v_i/u_i$: one division. Then $\cos(\mathbf{u}, \mathbf{v}) = +1$ if $\alpha > 0$ and -1 if $\alpha < 0$.

Evaluating the definition instead would cost one dot product, two norms, one mult and one division: $(2n - 1) + 2n + 2n + 1 + 1 = 6n + 1 = 151$.

2. How costs scale: asymptotic analysis

Section 1 produced exact counts: n , $2n - 1$, $2n$, $5n - 1$. Exactness is satisfying but it is rarely what we want to know. In computer science the question is almost always how the count *scales* — what happens to the cost as the input grows.

2.1. What an algorithm is

Definition 2.1 (Algorithm, informally). An **algorithm** is a finite sequence of steps aimed at solving a problem. It usually has an **input** and always produces an **output**.

Vector addition is about as simple an algorithm as exists. Its input is the pair $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$; its steps are the n componentwise adds; its output is the vector $\mathbf{w} = \mathbf{u} + \mathbf{v}$.

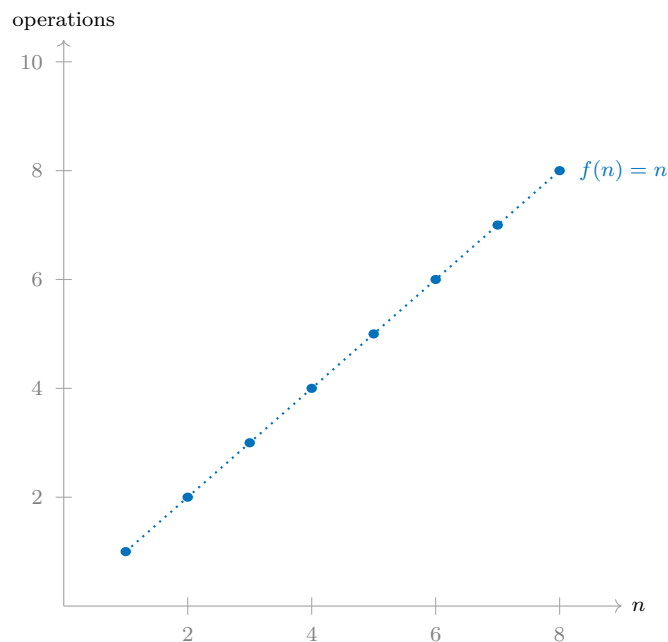
Remark. Every algorithm emits an output — an algorithm that produced nothing would have solved nothing. An input, though, is optional: an algorithm that prints the first thousand primes takes no input at all, and simply runs.

2.2. Operation count as a function of input size

To speak about scaling we need to say what is growing. For an algorithm on vectors, the **input size** is the number of components, n . So write the operation count as a function of it:

$$f(n) = n,$$

where n is the number of components and $f(n)$ is the exact number of operations vector addition performs. Note that n ranges over the natural numbers — an input size is a count, and there is no vector in $\mathbb{R}^{2.7}$ — so the graph of f is a set of isolated points, not a curve.

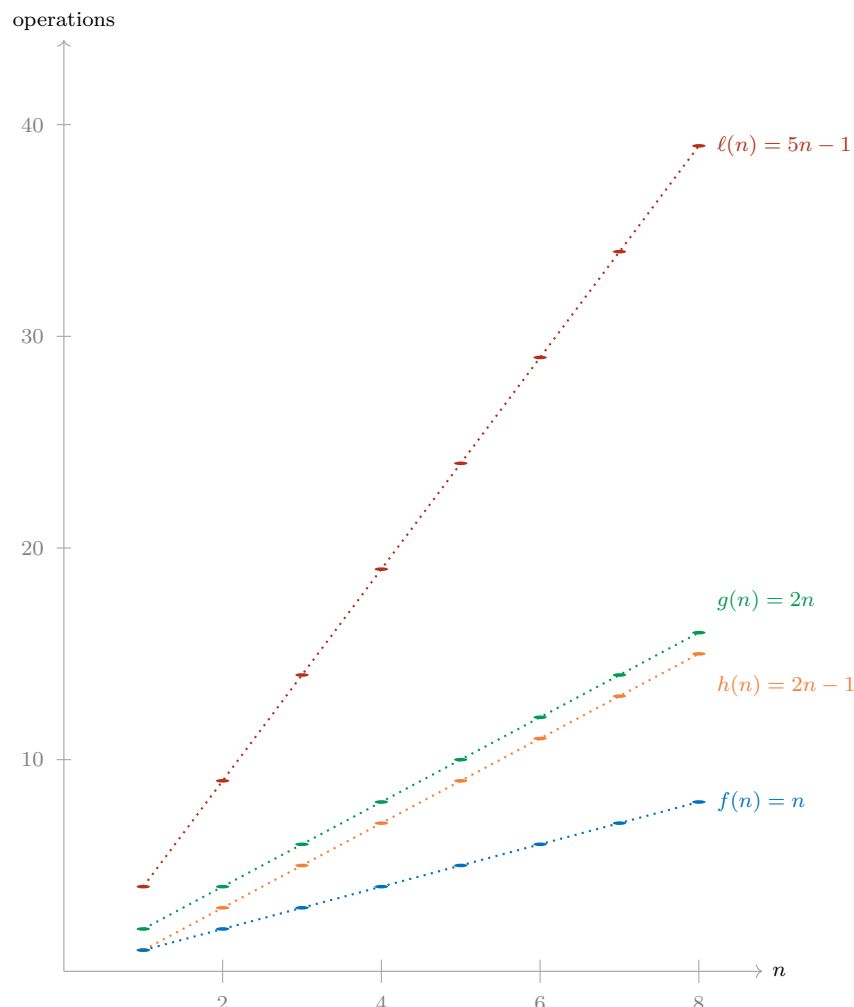


The dotted line is interpolation, drawn only so the eye can see the pattern; the algorithm exists only at the dots. That pattern — the relative geometry of f — is what we mean by the **scale** of the algorithm, and here it is plainly *linear*. The number of operations vector addition performs scales linearly with the number of components.

2.3. The other algorithms

We already have counts for three more. Writing each as a function of input size:

$$\underbrace{g(n) = 2n}_{\|\mathbf{u}\|}, \quad \underbrace{h(n) = 2n - 1}_{\langle \mathbf{u}, \mathbf{v} \rangle}, \quad \underbrace{\ell(n) = 5n - 1}_{\text{proj}_{\mathbf{u}}(\mathbf{v}) \text{ computed efficiently}}.$$



What do the four have in common? Every one of them is a straight line. The slopes differ and one is shifted down by 1, but the *shape* is identical: all four algorithms scale linearly with the number of components. Make the input as large as you like and that does not change — a line stays a line.

2.4. Big-O notation

Since the shape is what matters and the constants are not, we want notation that records the shape and discards the rest.

For a count that scales linearly in the long run we write $O(n)$, read “big-oh of n ”, and call it the algorithm’s **runtime complexity** — also its **asymptotic runtime** or **time complexity**.

So vector addition is $O(n)$. And so are the norm, the dot product, and the projection, despite their different constants and shifts. Big-O is concerned only with the underlying scale, which is set by the **dominant term** of the count.

A common confusion (We are not measuring time). “Time complexity” is a misleading name, and it is worth being clear about it early. We are counting *operations*, not seconds. How long a program actually takes depends on the processor, the language, the compiler, how the data sits in memory,

and what else the machine is doing — two $O(n)$ implementations can easily differ by a factor of a hundred in wall-clock time. What Big-O describes is how the operation count *grows*, and that is a property of the algorithm rather than of any machine that runs it.

2.5. Finding the dominant term

For a polynomial count the dominant term is the highest-degree one. Suppose an algorithm's count is modelled by

$$f(n) = 3n^2 + 2n + 3.$$

As n grows, $3n^2$ outpaces $2n$ and 3 so completely that they stop mattering, and the coefficient 3 does not change the shape — a parabola stretched is still a parabola. So f scales quadratically and we write $f(n)$ is $O(n^2)$.

When the terms are not both polynomials it can be less obvious which one wins, and a table of values helps.

Example 2.1 (A table that nearly misleads). Consider $f(n) = 2^n + n^5$.

n	2^n	n^5	larger
1	2	1	2^n
2	4	32	n^5
5	32	3,125	n^5
10	1,024	100,000	n^5
15	32,768	759,375	n^5
20	1,048,576	3,200,000	n^5
22	4,194,304	5,153,632	n^5
23	8,388,608	6,436,343	2^n
25	33,554,432	9,765,625	2^n
30	1,073,741,824	24,300,000	2^n

Read only the first six rows and you would conclude that n^5 dominates. It does not. The two cross between $n = 22$ and $n = 23$, and past that 2^n pulls away without limit — by $n = 30$ it is more than forty times larger, and the gap keeps widening. So the dominant term is 2^n , and

$$f(n) = 2^n + n^5 \quad \text{is} \quad O(2^n).$$

A common confusion. That example is the reason a table of values is a hint and not a proof. An exponential eventually beats every polynomial, no matter how large the polynomial's degree — but “eventually” can be far to the right of where you stopped looking. Big-O is a statement about the long run, so a handful of small values can point the wrong way.

Remark (A tip: compare with a ratio). A quick way to settle which of two positive functions dominates is to look at their ratio as n grows. If

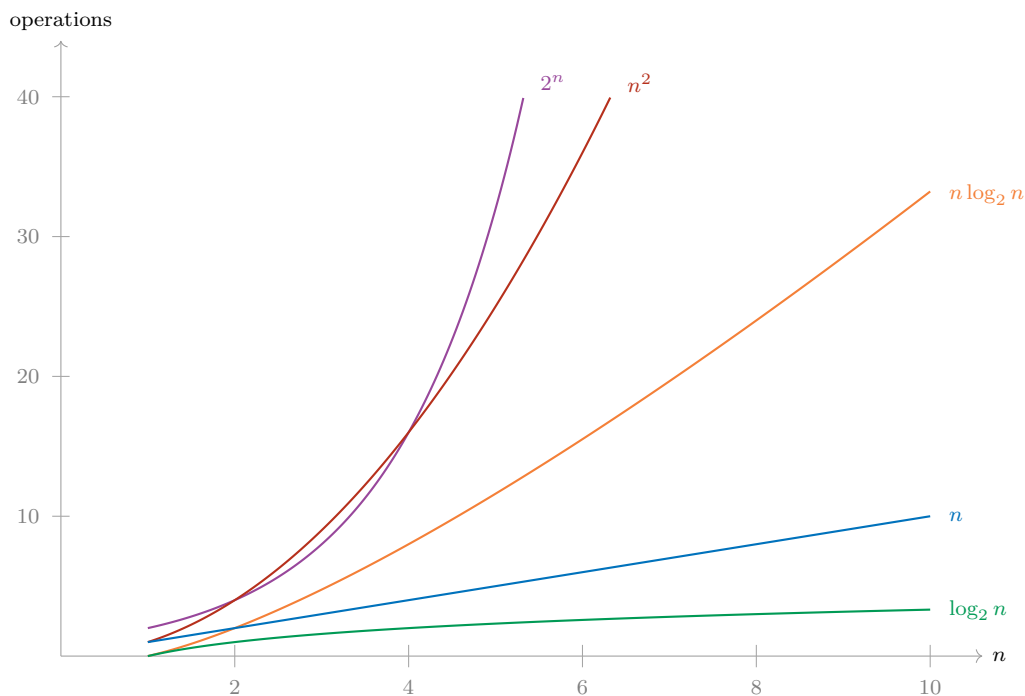
$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0,$$

then f is negligible against g , so g is the dominant one; if the limit is ∞ , then f dominates; and if it is a positive constant, the two scale alike. Two standard facts from Calculus I settle most cases immediately: exponentials outgrow all polynomials, and all polynomials outgrow logarithms. For the example above, $n^5/2^n \rightarrow 0$, confirming 2^n as dominant without any table.

2.6. Complexities you will meet

A few growth rates account for most algorithms in practice.

Complexity	A typical algorithm
$O(1)$	reading one entry of a vector
$O(\log_2 n)$	binary search in a sorted index
$O(n)$	vector addition, the dot product, the norm
$O(n \log_2 n)$	a good general-purpose sort
$O(n^2)$	a naive sort; a matrix–vector product
$O(n^3)$	naive matrix multiplication; Gaussian elimination
$O(2^n)$	checking every subset of n items



The vertical spread at the right-hand edge is the whole point. At $n = 10$ the logarithm has reached about 3 while 2^n passed 40 before n even reached 6. Choosing a better algorithm is not a small optimisation like the ones in §1.6; it changes which problems are solvable at all.

2.7. The formal definition

Everything so far has been informal — “dominant term”, “in the long run”. Here is the precise statement.

Definition 2.2 (Big-O). For functions f and g , we say $f(n)$ is $O(g(n))$ if and only if there exist constants $C > 0$ and $n_0 \geq 0$ such that

$$|f(n)| \leq C |g(n)| \quad \text{for all } n \geq n_0.$$

Every function we will use counts operations, so all of them are positive, and the absolute values may be dropped:

$$f(n) \leq C g(n) \quad \text{for all } n \geq n_0.$$

Read the definition as licensing two concessions. The constant C says we do not care about a fixed multiplicative factor; the threshold n_0 says we do not care what happens for small inputs. Proving f is $O(g)$ therefore amounts to *exhibiting* a C large enough and an n_0 far enough right that the inequality holds from there on. You do not need the smallest such C — any one that works will do.

2.8. Three proofs

Example 2.2. Show that $f(n) = 2n^2 + 2n$ is $O(n^2)$.

For every $n \geq 1$ we have $n \leq n^2$, hence $2n \leq 2n^2$, and so

$$f(n) = 2n^2 + 2n \leq 2n^2 + 2n^2 = 4n^2.$$

Take $C = 4$ and $n_0 = 1$. ■

Example 2.3. Show that $f(n) = n^4 + n^3$ is $O(n^4)$.

For every $n \geq 1$ we have $n^3 \leq n^4$, so

$$f(n) = n^4 + n^3 \leq n^4 + n^4 = 2n^4.$$

Take $C = 2$ and $n_0 = 1$. ■

Both follow the same recipe: bound every lower-order term by the dominant one, then collect. That recipe handles any polynomial.

Example 2.4 (Challenge). Show that $1^2 + 2^2 + 3^2 + \cdots + n^2$ is $O(n^3)$.

Here there is no highest-degree term to bound against, because the number of terms itself grows with n . The trick is to bound each term by the largest one. Every index k in the sum satisfies $k \leq n$, hence $k^2 \leq n^2$; and there are exactly n terms. Therefore

$$\sum_{k=1}^n k^2 \leq \underbrace{n^2 + n^2 + \cdots + n^2}_{n \text{ terms}} = n \cdot n^2 = n^3.$$

Take $C = 1$ and $n_0 = 1$. ■

2.9. A note on what Big-O does and does not say

Complexity notation comes in several forms. Big-O is an *upper* bound on growth — it says the cost is no worse than a certain shape. There are companions for the other directions: Ω for a lower bound, and Θ when upper and lower bounds match. For this course Big-O is enough.

Remark. Big-O is often described loosely as “the worst case”, and for our purposes nothing goes wrong in saying so. Strictly, though, two separate ideas are being run together. *Worst, best* and *average case* concern which *input* you measure; Big-O concerns how a count *grows* once you have chosen. One can perfectly well speak of an average-case $O(n \log_2 n)$.

For the algorithms in §1 the distinction happens not to arise: vector addition performs exactly n adds whatever the entries happen to be, and the same holds for the dot product, the norm and the projection. Their cost depends only on n , so best, average and worst case coincide. Algorithms where they differ — searching, sorting, elimination with pivoting — are where the vocabulary earns its keep.

Big-O is ubiquitous well beyond this course: it is the common language of software engineering, numerical analysis and scientific computing, and it is how the cost of every algorithm in the rest of this unit will be reported.

Looking ahead. One assumption remains unexamined. Throughout §1 and §2 we have counted operations on *real numbers*, as though a machine could hold one. It cannot: there are infinitely many reals and only finitely many patterns of bits. What a computer actually stores is a *floating-point* number, and arithmetic on those is not quite the arithmetic of $\mathbb{R}$ — sums can depend on the order you add them, and quantities that should cancel exactly do not. That is the next topic, and the last of this lecture.

3. The set a computer actually works in

3.1. Every subject starts with a set

Mathematical subjects share a structural pattern, and it is worth naming before we disturb it. In each one there is a *set* of objects and a collection of *operations* or rules that act on them. Everything else — the questions, the observations, the conjectures, the theorems — grows out of asking what happens to those objects under those rules in one scenario after another.

Calculus is a clear case. The underlying set is the real numbers, together with functions built from them. The operations on functions are governed by properties of $\mathbb{R}$, and out of those properties comes the limit; out of the limit comes the derivative; and out of the derivative comes the rest of the subject. Nothing in that chain was available until the set was fixed.

Underlying set	Some operations	What grows from it
$\mathbb{R}$, and functions on it	limits, derivatives, integrals	calculus, real analysis
$\mathbb{C}$	complex arithmetic, conjugation	complex analysis
$\mathbb{Z}$	divisibility, congruence, counting	number theory, combinatorics
$\mathbb{R}^n$, later matrices	addition, scaling, the dot product	linear algebra

The underlying set plants the seed for a whole garden of mathematical knowledge.

Remark. The vector unit was exactly this pattern, carried out in front of you. We fixed the set — $\mathbb{R}^2$, later $\mathbb{R}^n$ — then fixed the operations, and then spent five parts asking what followed: when do two vectors span, when are they independent, what does orthogonality buy, which subsets are closed. None of those questions could even be *stated* before the set and the operations were on the table.

3.2. A computer works in a different set

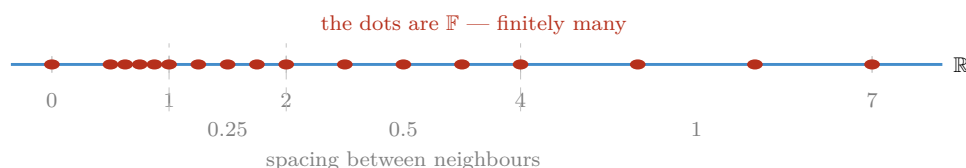
Here is the disturbance. When a computer tackles a mathematical problem — and especially a *continuous* one, a problem posed over the real numbers — it does not work in $\mathbb{R}$. It works in a peculiar set of its own.

Definition 3.1 (Floating-point numbers, informally). The **floating-point numbers**, written $\mathbb{F}$, are a *finite* set of numbers used to approximate the real numbers, each carrying only a fixed, finite number of digits.

Two consequences follow immediately, and both matter.

- $\mathbb{F}$ is *finite*, while $\mathbb{R}$ is not merely infinite but uncountable. So $\mathbb{F} \subset \mathbb{R}$: every floating-point number is a real number, and almost no real number is a floating-point number. The machine is working inside a very sparse subset of the set the problem was posed in.
- The elements of $\mathbb{F}$ are *not evenly spaced*. They crowd together near zero and spread apart as the magnitude grows, because a fixed number of digits buys finer resolution for small numbers than for large ones.

The picture below shows both facts at once. It uses a miniature floating-point system — three digits of precision in base 2, with only four exponents — so that the whole set fits on the page; a real system has vastly more points but is spaced in exactly this pattern.



Read the spacing across the picture: it *doubles* at every power of two. Between 1 and 2 the representable numbers are 0.25 apart; between 2 and 4 they are 0.5 apart; between 4 and 8 they are 1 apart. Near zero they are packed tightly, and far from zero they are strewn thinly. The line is continuous; the dots are all the machine has.

Everything in $\mathbb{R}$ that is not a dot must be *rounded* to the nearest dot before the machine can hold it. That rounding is small, and for a single number it is harmless. The question — which is what makes this a topic rather than a footnote — is what happens when millions of such roundings accumulate through an algorithm like the ones we counted in §1.

3.3. Floats and doubles

You have almost certainly met this set already. Programming languages expose it directly, and name the members by their **precision** — the number of digits each one carries:

Name	Storage	Roughly
float, single precision, float32	32 bits	about 7 decimal digits
double, double precision, float64	64 bits	about 16 decimal digits

A `double` is the default nearly everywhere it matters: it is what Python’s `float` is, what MATLAB uses unless told otherwise, and what numerical libraries assume. A `float` halves the storage and the precision, which is a real trade in graphics and machine learning and a liability almost everywhere else.

The word *precision* is the one to hold on to. It does not mean the numbers are approximately right in some vague sense; it means each one carries a specific, countable number of digits, and everything past that is gone.

3.4. Base 2

One more ingredient before we can say what a floating-point number actually *is*. A classical computer stores everything — numbers included — as patterns of 0s and 1s, which is to say in **binary**.

Binary is not exotic. It is simply a numerical *base*, base 2, and we have been conditioned by a decade of schooling to think in base 10 instead — for no better reason than that people have ten fingers. In any base b , a string of digits is shorthand for a sum of multiples of powers of b :

	Place values	Value
237_{10}	$2(10^2) + 3(10^1) + 7(10^0)$	$200 + 30 + 7 = 237$
1011_2	$1(2^3) + 0(2^2) + 1(2^1) + 1(2^0)$	$8 + 0 + 2 + 1 = 11$

The same idea continues to the right of the point, with negative powers:

$$0.101_2 = 1(2^{-1}) + 0(2^{-2}) + 1(2^{-3}) = \frac{1}{2} + \frac{1}{8} = 0.625.$$

So the toy system in the figure above was nothing more than this: a few binary digits, scaled by a power of two.

Looking ahead. A base is a choice of bookkeeping, and normally an innocent one — 0.625 is 0.625 whether you write it in base 10 or as 0.101_2 . But the choice is not always innocent, because a fraction that terminates in one base need not terminate in another. That is where the surprises live, and it is where we pick up next.

3.5. From base 10 to base 2

Reading a binary string is the easy direction: multiply out the place values, as above. Producing one is the direction we need, and it takes an algorithm. Split the number at the point and treat the two halves separately, because they obey different rules.

The integer part: repeated division by 2

Divide by 2, record the remainder, replace the number by the quotient, and repeat until the quotient is 0. The remainders, read *bottom to top*, are the binary digits.

Example 3.1. Convert 13_{10} to base 2.

Division	Quotient	Remainder
$13 \div 2$	6	1 $\leftarrow$ read last
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1 $\leftarrow$ read first

Reading upward, $13_{10} = 1101_2$. Check: $8 + 4 + 0 + 1 = 13$. ■

The algorithm is not a trick. The first remainder answers “is this number odd?”, which is exactly the 2^0 digit; dividing by 2 then shifts the remaining digits one place right so that the next one can be asked the same question.

The fractional part: repeated multiplication by 2

Multiply by 2. The integer part of the result — necessarily 0 or 1 — is the next binary digit. Discard it, keep the fraction that remains, and repeat. The digits, read *top to bottom*, are the binary expansion. Stop when the fraction reaches 0, or when you have as many digits as you can store.

Example 3.2. Convert 0.6875_{10} to base 2.

Product	Result	Digit	Carry forward
0.6875×2	1.375	1 $\leftarrow$ read first	0.375
0.375×2	0.75	0	0.75
0.75×2	1.5	1	0.5
0.5×2	1.0	1 $\leftarrow$ read last	0 (stop)

Reading downward, $0.6875_{10} = 0.1011_2$. Check: $\frac{1}{2} + \frac{1}{8} + \frac{1}{16} = 0.6875$. ■

Putting the halves back together,

$$13.6875_{10} = 1101.1011_2,$$

and we will encode exactly this number into hardware in §3.8.

When the algorithm does not stop

The integer algorithm always terminates — the quotient strictly decreases and must reach 0. The fractional one need not.

Example 3.3. Convert 0.1_{10} to base 2.

Product	Result	Digit	Carry forward
0.1×2	0.2	0	0.2
0.2×2	0.4	0	0.4
0.4×2	0.8	0	0.8
0.8×2	1.6	1	0.6
0.6×2	1.2	1	0.2 $\leftarrow$ seen at row 2

The carry has returned to a value it already took, so from here the work repeats forever, the block 0011 recurring without end:

$$0.1_{10} = 0.0\overline{0011}_2 = 0.000110011001100\dots_2.$$

■

This is the phenomenon flagged at the end of §3.4, and the rule behind it is worth stating. A fraction p/q in lowest terms terminates in base b exactly when every prime factor of q divides b . Base 10 has prime factors 2 and 5; base 2 has only 2. So $1/10$ terminates in decimal and cannot terminate in binary, because of the factor of 5 that base 2 has no way to absorb.

A common confusion ($0.1 + 0.2 \neq 0.3$). Type `0.1 + 0.2` into Python, or JavaScript, or C, and you get `0.30000000000000004`. This is not a bug in the language, and every language does it. Neither `0.1` nor `0.2` is a floating-point number — by the example above, neither has a finite binary expansion — so the machine stores the nearest float to each. It then adds those two stored values exactly and rounds again. The rounding of a sum need not equal the sum of the roundings, and here it does not. The consequence for you is a habit: never test two computed floating-point quantities for equality with `==`. Test whether they differ by less than a tolerance you have chosen on purpose.

3.6. Normalized form

A number has many representations as a digit string times a power of two:

$$1101.1011_2 = 1101.1011_2 \times 2^0 = 110.11011_2 \times 2^1 = 1.1011011_2 \times 2^3 = 0.11011011_2 \times 2^4,$$

all equal, all shifting the point against a matching change of exponent. Storing a number in hardware requires picking one of them, once and for all.

Definition 3.2 (Normalized form). A nonzero binary number is in **normalized form** when it is written

$$\pm 1.f_1f_2f_3\cdots \times 2^e,$$

that is, with exactly one nonzero digit to the left of the point. The full quantity $1.f_1f_2\cdots$ is the **significand**; the digits $f_1f_2\cdots$ after the point are the **mantissa**, or fraction; and e is the **exponent**.

To normalize, shift the point until it sits immediately after the leading 1. The exponent is the number of places shifted: positive if you moved left, negative if you moved right.

As given	Shift	Normalized
1101.1011 ₂	3 places left	$1.1011011_2 \times 2^3$
0.0001101 ₂	4 places right	$1.101_2 \times 2^{-4}$
1 ₂	none	$1.0_2 \times 2^0$
10.0 ₂	1 place left	$1.0_2 \times 2^1$

Now look at what every entry in the right-hand column has in common. The digit before the point is 1. It is 1 in every row, and it must be: base 2 has exactly one nonzero digit, and normalized form demands that the leading digit be nonzero. A quantity that is the same in every case carries no information, and there is no reason to spend storage on it. That observation is worth a full bit, and §3.8 collects it.

Remark. Zero is the exception, and it has to be. It has no nonzero leading digit, so it has no normalized form at all, and no choice of e produces it. It is handled by a reserved bit pattern instead — §3.10.

3.7. A standard: IEEE 754

Normalized form tells us *what* to store: a sign, a mantissa and an exponent. It does not say how many bits each gets, in what order, how to round what will not fit, or what to do when a result is too large to represent. Before the mid-1980s each manufacturer answered those questions its own way. The same program, compiled on two machines, could return two different numbers, and there was no way to say which was correct.

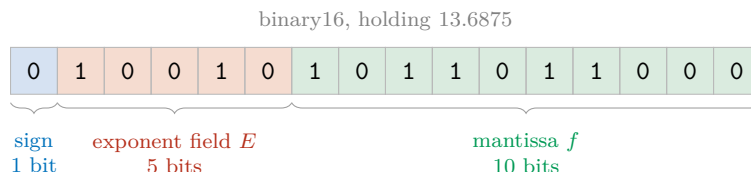
The **Institute of Electrical and Electronics Engineers** — the IEEE, pronounced “eye-triple-E” — ended that with the IEEE 754 standard, first published in 1985 and revised since. It fixes the formats, the rounding rules and the behaviour on overflow and on invalid operations. Essentially every processor and every language you will use implements it, so “floating point” in practice means IEEE 754. The rest of this section is a description of that standard.

IEEE name	Common name	Width	Typical use
binary16	half precision	16 bits	machine learning, graphics
binary32	single precision, float	32 bits	graphics, embedded, ML
binary64	double precision, double	64 bits	the default nearly everywhere
binary128	quadruple precision	128 bits	specialised high-accuracy work

We will work with binary16 and binary32, because they are short enough to write out in full on the page. Everything said about them holds verbatim for binary64 with three numbers changed.

3.8. The three fields, and the hidden 1

A floating-point number is one bit string, cut into three consecutive fields.



Write s for the sign bit, E for the exponent field read as an unsigned integer, and f for the mantissa read as the digits after a binary point. For a *normal* number the value stored is

$$x = (-1)^s \times 1.f \times 2^{E-\text{bias}}$$

with the bias a fixed constant of the format, discussed in §3.9.

Format	Total	Sign	Exponent k	Mantissa m	Bias
binary16	16	1	5	10	15
binary32	32	1	8	23	127
binary64	64	1	11	52	1023

The hidden 1

Notice what the formula does and the bit string does not. The value is built from $1.f$, but only f is stored. The leading 1 is never written down anywhere in the sixteen bits — the hardware supplies it when decoding. It is called the **hidden bit**, or the implicit leading bit, and §3.6 is the reason it can be omitted: in normalized binary the leading digit is always 1, so writing it would be storing a constant.

Definition 3.3 (Precision). The **precision** p of a binary floating-point format is the number of significand digits, counting the hidden bit. If the mantissa field holds m bits then

$$p = m + 1.$$

So binary16 has $p = 11$ from a 10-bit field, binary32 has $p = 24$ from 23 bits, and binary64 has $p = 53$ from 52. The hidden bit is a free doubling of the number of representable values, at no storage cost — and it is available only because we are in base 2. In base 10 the leading digit could be any of 1 through 9, and would have to be written down.

Example 3.4 (Encoding). Store 13.6875 in binary16.

1. *Sign.* The number is positive, so $s = 0$.
2. *Convert.* By §3.5, $13.6875 = 1101.1011_2$.
3. *Normalize.* By §3.6, $1101.1011_2 = 1.1011011_2 \times 2^3$, so $e = 3$ and the mantissa digits are 1011011.
4. *Bias the exponent.* $E = e + 15 = 18 = 10010_2$, five bits.
5. *Pad the mantissa* on the right to ten bits: 1011011000.

The stored string is

$$\underbrace{0}_s \underbrace{10010}_{E=18} \underbrace{1011011000}_f,$$

which is the string in the diagram above. In binary32 the same number becomes

$$\underbrace{0}_s \underbrace{10000010}_{E=130} \underbrace{101101100000000000000000}_f,$$

the exponent field now holding $3 + 127 = 130$ and the mantissa padded to 23 bits. ■

Example 3.5 (Decoding). What binary16 number is 1 10001 0100000000?

The sign bit is 1, so the number is negative. The exponent field reads $10001_2 = 17$, and unbiasing gives $e = 17 - 15 = 2$. Restoring the hidden bit, the significand is $1.01_2 = 1.25$. Hence

$$x = (-1)^1 \times 1.25 \times 2^2 = -5. \quad \blacksquare$$

3.9. The bias, and the range of the exponent

Exponents must be allowed to be negative — that is what represents numbers smaller than 1. There are two ways to store a signed integer in a fixed field, and IEEE 754 chose the simpler one: store not e but $e + \text{bias}$, an unsigned integer, where for a k -bit exponent field

$$\text{bias} = 2^{k-1} - 1.$$

For $k = 5$ that is 15, for $k = 8$ it is 127, for $k = 11$ it is 1023 — the constants tabulated in §3.8. Decoding just subtracts: $e = E - \text{bias}$.

Two exponent fields are not available for this, because they are reserved to mark the special values of §3.10: the all-zeros field $E = 0$ and the all-ones field $E = 2^k - 1$. So a normal number has E somewhere in $1 \leq E \leq 2^k - 2$, and subtracting the bias gives the range of usable exponents.

Format	Stored E	Exponent $e = E - \text{bias}$	Range of e
binary16	1...30	$E - 15$	$-14 \leq e \leq 15$
binary32	1...254	$E - 127$	$-126 \leq e \leq 127$
binary64	1...2046	$E - 1023$	$-1022 \leq e \leq 1023$

Remark. Why bias rather than a sign bit on the exponent? Because biasing makes the exponent fields sort in the same order as the exponents they denote. A consequence is that two positive floats can be compared simply by comparing their bit strings as unsigned integers — the hardware needs no special floating-point comparison circuit at all. That the sign bit comes first, the exponent second and the mantissa last is part of the same design: it puts the fields in decreasing order of significance.

3.10. The reserved patterns: zero, infinity and NaN

The two reserved exponent fields, combined with whether the mantissa is zero, give four cases.

Exponent field E	Mantissa f	Meaning	Value
all zeros	$f = 0$	signed zero	± 0
all zeros	$f \neq 0$	subnormal	$(-1)^s 0.f \times 2^{1-\text{bias}}$
$1 \dots 2^k - 2$	any	normal	$(-1)^s 1.f \times 2^{E-\text{bias}}$
all ones	$f = 0$	infinity	$\pm \infty$
all ones	$f \neq 0$	not a number	NaN

In binary16, for instance,

$$0 \text{ 11111 0000000000} = +\infty, \quad 1 \text{ 11111 0000000000} = -\infty, \quad 0 \text{ 11111 1000000000} = \text{NaN}.$$

Infinity: $\pm \infty$

Infinity is produced by *overflow* — a result whose magnitude exceeds the largest representable number — and by division of a nonzero number by zero. It is a value, not a crash: the computation continues, with $\infty + 1 = \infty$ and $1/\infty = 0$. Two infinities are needed because the two directions of overflow must be distinguished.

Subnormals

The pattern $E = 0$ with $f \neq 0$ drops the hidden bit and reads the significand as $0.f$, which fills in the gap between zero and the smallest normal number with a few evenly spaced values. Without them there would be a sudden cliff at the bottom of the range, and two distinct small numbers could have a difference that rounds to zero even though they are not equal.

Not a number: NaN

NaN stands for *not a number*, and it is what an operation returns when no numerical answer is defensible: $0/0$, $\infty - \infty$, ∞/∞ , $\sqrt{-1}$. Two properties make it worth knowing about.

- NaN *propagates*. Any arithmetic operation with a NaN input returns NaN. One of them entering a long computation contaminates everything downstream of it.
- NaN is not equal to itself. The comparison $\text{NaN} = \text{NaN}$ is false — it is the only value in the system for which $x = x$ fails.

A *common confusion* (A vector of NaNs). The second property is deliberate, and it is how you test for one: the idiom $\mathbf{x} \neq \mathbf{x}$ is true exactly when $\mathbf{x}$ is NaN. The first property is why, when a numerical program goes wrong, the symptom is so often an output vector every entry of which is NaN. The final state tells you nothing about the cause; a single division by zero thousands of operations earlier will produce it. Finding where the contamination started, rather than where it surfaced, is the work.

3.11. The range of a floating-point system

With the exponent range from §3.9 we can say exactly how large and how small the members of $\mathbb{F}$ are. The largest magnitude pairs the largest exponent $e_{\max} = \text{bias}$ with the largest significand, which is $1.11 \cdots 1_2$ with $p-1$ ones, namely $2 - 2^{1-p}$. The smallest positive normal magnitude pairs the smallest exponent $e_{\min} = 1 - \text{bias}$ with the smallest significand, $1.00 \cdots 0_2 = 1$. So

$$x_{\max} = (2 - 2^{1-p}) 2^{e_{\max}}, \quad x_{\min} = 2^{e_{\min}}.$$

Format	Smallest positive normal	Largest	Precision p
binary16	$2^{-14} \approx 6.10 \times 10^{-5}$	65504	11
binary32	$2^{-126} \approx 1.18 \times 10^{-38}$	$\approx 3.40 \times 10^{38}$	24
binary64	$2^{-1022} \approx 2.23 \times 10^{-308}$	$\approx 1.80 \times 10^{308}$	53

Anything larger in magnitude than $x_{\max}$ overflows to $\pm\infty$; anything smaller than the smallest subnormal underflows to ± 0 . And note which field controls which: the *exponent* bits buy range, the *mantissa* bits buy precision, and they are two independent budgets. Going from binary16 to binary32 spends 3 more bits on range and 13 more on precision.

Remark. Binary16's ceiling of 65504 is low enough to hit by accident, which is exactly what happens when neural networks are trained in half precision for speed: intermediate quantities overflow to ∞ , the ∞ turns into a NaN at the next subtraction, and training collapses. The standard remedy — multiplying through by a constant to move the computation into range, then dividing it out afterwards — is called loss scaling, and it is a working engineer's response to the table above.

Looking ahead. We now know precisely which numbers a machine can hold. The remaining question is the one that matters for computation: what happens to everything else. How far apart are the dots, how much is lost when a real number is pushed to the nearest one, and what does that do to an algorithm that performs the operation five thousand times?

4. Precision, spacing and error

4.1. Machine precision

Start at 1 and ask for the next floating-point number. In normalized form $1 = 1.00 \dots 0_2 \times 2^0$, and the smallest change we can make is to set the last mantissa bit, giving $1.00 \dots 01_2 \times 2^0 = 1 + 2^{-(p-1)}$. There is nothing in between, because there is no bit left to change.

Definition 4.1 (Machine precision). The **machine precision**, or machine epsilon, of a binary floating-point system of precision p is

$$\varepsilon_{\text{mach}} = 2^{1-p},$$

the distance from 1 to the next larger floating-point number.

Format	p	$\varepsilon_{\text{mach}} = 2^{1-p}$	Decimal digits
binary16	11	$2^{-10} \approx 9.77 \times 10^{-4}$	about 3
binary32	24	$2^{-23} \approx 1.19 \times 10^{-7}$	about 7
binary64	53	$2^{-52} \approx 2.22 \times 10^{-16}$	about 16

The last column is where the informal claim of §3.3 came from. A `float` was said to carry “about 7 decimal digits” and a `double` “about 16”; those are just $\varepsilon_{\text{mach}}$ written in scientific notation and the exponent read off.

A *common confusion* ($\varepsilon_{\text{mach}}$ is not the smallest positive number). These are two different quantities answering two different questions, and they differ by more than thirty orders of magnitude. In binary32 the smallest positive normal number is about 1.18×10^{-38} , while $\varepsilon_{\text{mach}}$ is about 1.19×10^{-7} . The first asks how small a magnitude the format can reach, and is set by the exponent field. The second asks how finely the format separates numbers near 1, and is set by the mantissa field. A number far smaller than $\varepsilon_{\text{mach}}$ is perfectly representable; what you cannot do is *add* it to 1 and see any change.

One companion constant, needed in §4.5. Half of the machine precision is called the **unit roundoff**,

$$u = \frac{1}{2}\varepsilon_{\text{mach}} = 2^{-p},$$

and it will turn out to be the largest relative error that rounding can commit.

4.2. Units in the last place

Machine precision measures the spacing near 1. Everywhere else the spacing is different, and §3.2 already showed the pattern in a picture. Here is the statement behind it.

Fix an exponent e and consider all the numbers with that exponent: they are $1.f \times 2^e$ with f running over every one of the $(p-1)$ -bit mantissa patterns. Consecutive ones differ by one unit in the last mantissa bit, which is $2^{-(p-1)}$ in the significand, hence $2^{-(p-1)} \times 2^e$ in value.

Definition 4.2 (Unit in the last place). Let x satisfy $2^e \leq |x| < 2^{e+1}$. The **unit in the last place** at x is the distance between consecutive floating-point numbers there:

$$\text{ulp}(x) = 2^{e-(p-1)} = 2^e \varepsilon_{\text{mach}}.$$

Read the formula twice, once for each factor.

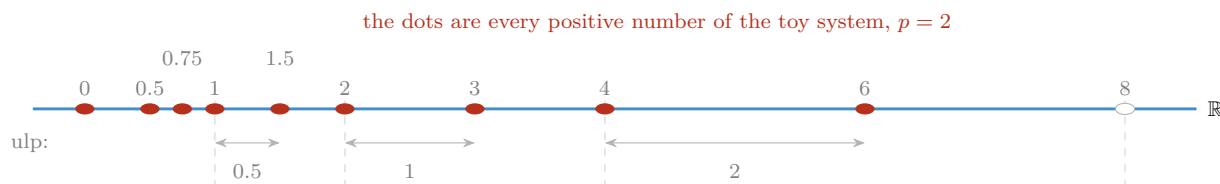
- The 2^e says the *absolute* spacing scales with the magnitude: double the number and you double the gap to its neighbour. This is the doubling visible in the figure of §3.2.
- The $\varepsilon_{\text{mach}}$ says the *relative* spacing does not change. Dividing through, $\text{ulp}(x)/|x|$ lies between $\varepsilon_{\text{mach}}/2$ and $\varepsilon_{\text{mach}}$ wherever x sits. The system carries the same number of significant digits at every scale.

The second point is what the name *floating point* refers to. The binary point is not nailed to a fixed column, as it is in a fixed-point system where all numbers are equally spaced; it floats to wherever the leading digit is, so that the available digits are always spent on the part of the number that matters.

A toy system, $p = 2$

Take base 2, precision $p = 2$ — one hidden bit and a single mantissa bit — and exponents $-1 \leq e \leq 2$. The significand is either $1.0_2 = 1$ or $1.1_2 = 1.5$, and that is all. Every positive number in this $\mathbb{F}$:

Significand		$e = -1$	$e = 0$	$e = 1$	$e = 2$
1.0_2	$= 1$	0.5	1	2	4
1.1_2	$= 1.5$	0.75	1.5	3	6
$\text{ulp} = 2^{e-1}$		0.25	0.5	1	2



The open circle at 8 is not a member of the toy system: it would need $e = 3$, and the exponent stops at 2. Check the definition against the picture. Between 1 and 2 the spacing is $\text{ulp} = 2^{0-1} = 0.5$; between 2 and 4 it is $2^{1-1} = 1$; between 4 and 8 it is $2^{2-1} = 2$. And the machine precision of this system is $\varepsilon_{\text{mach}} = 2^{1-2} = 0.5$, which is exactly the gap from 1 to the next number, 1.5. Small as it is, this system has every feature of binary64; it merely has 8 positive numbers instead of about 10^{308} .

4.3. Rounding to nearest

A real number that is not a dot must be moved to one. Which one is a policy decision, and IEEE 754 offers several *rounding modes*. One is the default everywhere, and it is the only one we will use.

Definition 4.3 (Round to nearest). The map $\text{round} : \mathbb{R} \rightarrow \mathbb{F}$ sends x to the floating-point number closest to it. When x falls exactly halfway between two neighbours, the tie is broken by choosing the one whose last mantissa bit is 0 — “round half to even”.

Example 4.1. In the toy system of §4.2:

- $\text{round}(2.6) = 3$. The neighbours are 2 and 3, at distances 0.6 and 0.4.

- $\text{round}(1.2) = 1$. The neighbours are 1 and 1.5, at distances 0.2 and 0.3.
- $\text{round}(2.5) = 2$. The neighbours 2 and 3 are both at distance 0.5, so this is a tie. Now $2 = 1.0_2 \times 2^1$ has last mantissa bit 0 and $3 = 1.1_2 \times 2^1$ has last mantissa bit 1, so the rule selects 2.

Remark. The tie rule looks fussy and is not. Always rounding halves upward would push every tie in the same direction, and in a sum of a million terms those pushes accumulate into a systematic drift. Sending half the ties down and half up lets them cancel instead.

The bound that matters follows immediately from §4.2. If x lies in the normalized range, it sits between two neighbours a distance $\text{ulp}(x)$ apart, and the nearer of the two can be no further away than half of that:

$$\boxed{|\text{round}(x) - x| \leq \frac{1}{2} \text{ulp}(x) = 2^{e-p}}$$

whenever $2^e \leq |x| < 2^{e+1}$. Everything in §4.5 is a consequence of that one line. Outside the normalized range the bound does not apply: above $x_{\max}$ rounding overflows to $\pm\infty$, and below the subnormal floor it underflows to ± 0 .

4.4. Absolute and relative error

We need vocabulary for how wrong an approximation is. Two measures, and they answer different questions.

Definition 4.4 (Absolute and relative error). Let $\hat{x}$ approximate a real number x . The **absolute error** is

$$E_{\text{abs}} = |\hat{x} - x|,$$

and, for $x \neq 0$, the **relative error** is

$$E_{\text{rel}} = \frac{|\hat{x} - x|}{|x|}.$$

Example 4.2. In the toy system, $\text{round}(2.6) = 3$. So

$$E_{\text{abs}} = |3 - 2.6| = 0.4, \quad E_{\text{rel}} = \frac{0.4}{2.6} \approx 0.1538.$$

Compare that relative error with the unit roundoff of this system, $u = 2^{-p} = 2^{-2} = 0.25$. As promised, $0.1538 \leq 0.25$. ■

Relative error is the one that travels. An absolute error of 0.4 is catastrophic in a quantity of size 1 and beneath notice in a quantity of size 10^6 ; the number alone does not say which situation you are in. Relative error is scale-free, and — by the second reading of ulp in §4.2 — it is precisely the quantity a floating-point system holds under control. The absolute spacing of $\mathbb{F}$ varies over three hundred orders of magnitude; the relative spacing never leaves a factor of two.

The same definitions for vectors

Nothing changes except the measuring stick. Where a scalar used absolute value, a vector uses a norm — the Euclidean norm of the vector unit, §2, will do.

Definition 4.5 (Error in a vector). Let $\hat{\mathbf{v}}$ approximate $\mathbf{v} \in \mathbb{R}^n$. Then

$$E_{\text{abs}} = \|\hat{\mathbf{v}} - \mathbf{v}\|, \quad E_{\text{rel}} = \frac{\|\hat{\mathbf{v}} - \mathbf{v}\|}{\|\mathbf{v}\|} \quad (\mathbf{v} \neq \mathbf{0}).$$

Both are single numbers describing the whole vector, not one number per entry. What they measure is the length of the *error vector* $\hat{\mathbf{v}} - \mathbf{v}$, which is exactly the picture the vector unit built: $\|\hat{\mathbf{v}} - \mathbf{v}\|$ is the distance between the two points.

Example 4.3. Take $\mathbf{v} = (1, 2.6)$ and round each entry into the toy system, giving $\hat{\mathbf{v}} = (1, 3)$. The error vector is $\hat{\mathbf{v}} - \mathbf{v} = (0, 0.4)$, so

$$E_{\text{abs}} = \|(0, 0.4)\| = 0.4, \quad E_{\text{rel}} = \frac{0.4}{\|(1, 2.6)\|} = \frac{0.4}{\sqrt{7.76}} \approx 0.1436.$$

Note that the first entry was stored exactly and contributed nothing; the relative error of the vector is not the relative error of its worst entry. ■

4.5. Floating-point arithmetic

Here is the difficulty the whole section has been building towards. Let $x, y \in \mathbb{F}$. Their exact sum $x + y$ is a real number, and there is no reason whatever for it to lie in $\mathbb{F}$ — in the toy system, 1.5 and 2 are both members and $1.5 + 2 = 3.5$ is not. So the machine *cannot* return $x + y$. What it returns instead is fixed by the standard.

Definition 4.6 (Floating-point arithmetic). For $x, y \in \mathbb{F}$ and an operation $*$ $\in \{+, -, \times, \div\}$, the corresponding machine operation is

$$\text{fl}(x * y) = \text{round}(x * y) :$$

form the exact result, then round it once to the nearest floating-point number. IEEE 754 requires this of all four arithmetic operations and of the square root.

This is a strong guarantee, and it is worth being clear about what it does and does not promise. It does not say the answer is correct. It says the answer is the *best possible* — the closest member of $\mathbb{F}$ to the true result — and that each operation commits at most one rounding. A machine that did anything less would be much harder to reason about.

Now we can quantify the damage.

Theorem 4.1 (Rounding as a relative perturbation). *Let x be any real number in the normalized range of a binary floating-point system with precision p . Then*

$$\text{round}(x) = x(1 + \delta) \quad \text{for some } \delta \text{ with } |\delta| < \varepsilon_{\text{mach}}.$$

Proof. If $x = 0$ then $\text{round}(x) = 0$ and $\delta = 0$ works, so assume $x \neq 0$ and let e be the exponent with $2^e \leq |x| < 2^{e+1}$. Define

$$\delta = \frac{\text{round}(x) - x}{x},$$

which is legitimate since $x \neq 0$, and rearranges at once to $\text{round}(x) = x(1 + \delta)$. It remains to bound δ . By the rounding bound of §4.3 the numerator satisfies $|\text{round}(x) - x| \leq \frac{1}{2} \text{ulp}(x) = 2^{e-p}$, and the denominator satisfies $|x| \geq 2^e$. Therefore

$$|\delta| = \frac{|\text{round}(x) - x|}{|x|} \leq \frac{2^{e-p}}{2^e} = 2^{-p} = \frac{1}{2} \varepsilon_{\text{mach}} < \varepsilon_{\text{mach}}. \quad \square$$

Remark. The proof delivers slightly more than the statement: it gives $|\delta| \leq 2^{-p} = u$, the unit roundoff of §4.1, which is half of $\varepsilon_{\text{mach}}$. Both forms are in common use, and the sharper one is the standard hypothesis in numerical analysis. Nothing in what follows depends on which you carry.

Combining the theorem with the definition of fl gives the statement that every error analysis in numerical linear algebra starts from: for $x, y \in \mathbb{F}$ and $*$ $\in \{+, -, \times, \div\}$,

$$\boxed{\text{fl}(x * y) = (x * y)(1 + \delta), \quad |\delta| \leq u.}$$

Read it in words: *every floating-point operation returns the exact answer, perturbed by a relative amount no larger than the unit roundoff*. One operation is harmless — in binary64 the perturbation is at the sixteenth digit. The subject begins when you chain N of them together, because those N perturbations may cancel or may compound, and which one happens is a property of the algorithm rather than of the hardware. An algorithm whose errors stay controlled is called *stable*, and one whose errors amplify is not; two mathematically identical formulas, like the two versions of the projection in §1.6, can differ in this respect as well as in cost.

A common confusion (Addition is not associative). In $\mathbb{R}$, $(a + b) + c = a + (b + c)$ always. In $\mathbb{F}$ it can fail. Work in binary16, where $\varepsilon_{\text{mach}} = 2^{-10}$, and take $a = 1$, $b = c = 2^{-11}$.

- *Left to right*. The exact sum $1 + 2^{-11}$ falls exactly halfway between 1 and $1 + 2^{-10}$, its two neighbours. Round half to even selects 1. Adding c to that repeats the same tie, so $\text{fl}(\text{fl}(a + b) + c) = 1$.
- *Right to left*. First $2^{-11} + 2^{-11} = 2^{-10}$, which is exactly representable, so no rounding occurs. Then $1 + 2^{-10}$ is exactly representable too, giving $\text{fl}(a + \text{fl}(b + c)) = 1 + 2^{-10}$.

The two orders disagree. The practical consequences are real: summing a long list of numbers gives different answers depending on the order, running the same sum in parallel gives a different answer again because the partial sums combine differently, and a compiler that reorders arithmetic to go faster changes your output. None of this is a defect in the hardware. It is what working in a finite set costs.

4.6. FLOPs

We can now close the loop back to §1. That section defined an *operation* as one addition, subtraction, multiplication, division or square root, and counted them on vectors in $\mathbb{R}^n$ — but $\mathbb{R}^n$ was a fiction, as this section has spent its length establishing. The operations were on $\mathbb{F}$ all along, and the standard name says so.

Definition 4.7 (FLOP). A **floating-point operation**, or **FLOP**, is a single arithmetic operation — one add, subtract, multiply, divide or square root — performed on floating-point numbers.

Nothing about the counting changes; only the name, which is now accurate. Restating the results of §1 in the standard vocabulary: for $\mathbf{u}, \mathbf{v} \in \mathbb{F}^n$,

Computation	Cost	Complexity
$\mathbf{u} + \mathbf{v}$	n FLOPs	$O(n)$
$\langle \mathbf{u}, \mathbf{v} \rangle$	$2n - 1$ FLOPs	$O(n)$
$\ \mathbf{u}\ $	$2n$ FLOPs	$O(n)$
$\text{proj}_{\mathbf{u}}(\mathbf{v})$	$5n - 1$ FLOPs	$O(n)$

And there is a second reading of the table now available. Each of those FLOPs, by §4.5, perturbs its result by a relative amount up to u . A dot product in $\mathbb{R}^{1000}$ performs 1999 of them, so it commits 1999 roundings on the way to a single number. Its cost and its accuracy are both consequences of the same count.

A *common confusion* (FLOPs and FLOPS are different things). The two are distinguished, unhelpfully, by the case of the final letter.

- **FLOPs** — lower-case *s*, a plural — counts operations. It is a property of an algorithm, and it is what §1 computed.
- **FLOPS** — upper-case *S*, for *per second* — is a rate. It is a property of hardware. A processor rated at 30 teraFLOPS performs up to 3×10^{13} floating-point operations every second.

Dividing one by the other estimates a running time,

$$\text{time} \approx \frac{\text{FLOPs}}{\text{FLOPS}},$$

which is optimistic — it assumes the processor never waits for data to arrive from memory, and in practice it often does — but it is the right first calculation, and it is why a FLOP count is worth having as a number and not only as a complexity class.

FLOP counts are the standard currency of scientific computing. The cost of solving a linear system, of factoring a matrix, of training a neural network, is quoted in FLOPs, and the capability of a machine is quoted in FLOPS. Both halves of this lecture were needed to make sense of either.

Looking ahead. Three tools have been assembled here: an exact operation count (§1), the growth rate that count belongs to (§2), and the arithmetic the count is actually performed in (§3–§4). From this point on, every algorithm we meet gets reported the same way — what it costs in FLOPs, what its complexity is, and what rounding does to its answer. The first of them will operate not on vectors but on matrices, and its cost will not be $O(n)$.